

Оптимизация моделей обработки естественного языка

Методы ускорения, сжатия и эффективного обучения NLP-моделей

HUGGING FACE

LORA

PEFT

QUANTIZATION

DISTILLATION

ONNX

FLASH ATTENTION

VLLM

TEXT AUGMENTATION

TOKENIZERS

TENSORRT-LLM

Тема 1. Зачем оптимизировать NLP-модели

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Современные модели обработки естественного языка (NLP) основаны на архитектуре Transformer (Vaswani et al., 2017) и содержат от десятков миллионов до сотен миллиардов параметров. BERT-base — 110M, GPT-2 — 1.5B, LLaMA-2 — 7–70B, GPT-4 — оценочно >1T. Обучение таких моделей с нуля требует тысяч GPU-часов и миллионов долларов.

NLP охватывает широкий спектр задач: классификация текста, NER (Named Entity Recognition), извлечение информации, машинный перевод, суммаризация, генерация текста, вопросно-ответные системы. Каждая задача предъявляет специфические требования к размеру модели, latency и качеству.

Ключевые проблемы: (1) Вычислительная стоимость — full fine-tuning BERT-large на одной GPU занимает часы, LLaMA-7B — дни. (2) Память — inference GPT-2 (1.5B) требует ~6 ГБ FP32, LLaMA-7B — ~28 ГБ. (3) Latency — Transformer имеет квадратичную сложность $O(n^2)$ по длине последовательности.

Семинар охватывает 8 методов оптимизации NLP: Transfer Learning, PEFT (LoRA, Adapters), Knowledge Distillation, Pruning, Quantization, Efficient Attention, Text Augmentation и Production Deployment.

Тема 1. Таксономия методов оптимизации NLP

ОБЗОР

Метод	Цель	Эффект	Инструменты
Transfer Learning	Переиспользование модели	×10–100 быстрее обучения	HuggingFace Transformers
PEFT (LoRA, Adapters)	Эффективное дообучение	0.1–1% обучаемых параметров	peft, LoRA, QLoRA
Knowledge Distillation	Сжатие модели	×2–6 меньше, 95–99% качества	DistilBERT, TinyBERT
Pruning	Удаление голов / слоёв	–30–50% параметров	nn.utils.prune, oBERT
Quantization	Снижение разрядности	×2–4 меньше, ×2–3 быстрее	bitsandbytes, GPTQ, AWQ
Efficient Attention	Ускорение Transformer	$O(n)$ вместо $O(n^2)$	Flash Attention, xformers
Text Augmentation	Расширение данных	+1–5% метрики	nlpaug, TextAttack
Deployment	Production inference	×3–10 ускорение	vLLM, TensorRT-LLM, ONNX

Тема 2. Transfer Learning для NLP

Transfer Learning в NLP реализуется через парадигму «pretrain → fine-tune». На первом этапе модель обучается на массивном неразмеченном корпусе (Wikipedia, Common Crawl, BookCorpus) задачам самосупервизированного обучения: Masked Language Model (BERT), Causal LM (GPT), Seq2Seq (T5).

На втором этапе модель дообучается (fine-tune) на целевой размеченной задаче с малым learning rate ($2e-5$ – $5e-5$). Ключевое преимущество: предобученная модель содержит «знание языка» — синтаксис, семантику, мировые знания — которое переносится на целевую задачу даже при малом объёме обучающих данных.

Стратегии fine-tuning: (1) Full fine-tuning — обновление всех параметров. Стандарт для BERT-base/large при достаточном объёме данных. (2) Feature extraction — замораживание Transformer, обучение только классификационной головы. Быстро, но качество ниже. (3) Gradual unfreezing (Howard & Ruder, 2018) — постепенное размораживание слоёв от верхних к нижним с убывающим learning rate. (4) PEFT — обучение лишь малой доли параметров (см. Тему 3).

Hugging Face Transformers — де-факто стандартная библиотека, предоставляющая доступ к 200 000+ предобученных моделей через единый API.

Тема 2. Fine-tuning BERT в Hugging Face

КОД

```
from transformers import (
    AutoTokenizer, AutoModelForSequenceClassification,
    TrainingArguments, Trainer)
from datasets import load_dataset
import evaluate # библиотека метрик HF

# Загрузка предобученного BERT и токенизатора
model_name = 'bert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=2) # бинарная классификация

# Загрузка и токенизация датасета (IMDB: sentiment analysis)
dataset = load_dataset('imdb')
def tokenize_fn(examples):
    return tokenizer(examples['text'],
        padding='max_length', truncation=True, max_length=256)
tokenized = dataset.map(tokenize_fn, batched=True)

# Настройка обучения
args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3, # число эпох
    per_device_train_batch_size=16,
    learning_rate=2e-5, # типичный LR для BERT
    weight_decay=0.01, # регуляризация
    evaluation_strategy='epoch',
    save_strategy='epoch')

trainer = Trainer(model=model, args=args,
    train_dataset=tokenized['train'],
    eval_dataset=tokenized['test'])
trainer.train() # запуск fine-tuning
```

Тема 2. Задача А: классификация тональности

📌 ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Fine-tune BERT-base на SST-2 (Stanford Sentiment). Сравните accuracy при full fine-tuning (3 эпохи) и feature extraction (замороженный encoder).

```
from transformers import AutoModelForSequenceClassification
from datasets import load_dataset
```

```
# Загрузка SST-2 (бинарный sentiment)
dataset = load_dataset('glue', 'sst2')
tokenized = dataset.map(lambda x: tokenizer(
    x['sentence'], padding='max_length',
    truncation=True, max_length=128), batched=True)
```

```
# Вариант 1: Full fine-tuning (все параметры обучаемы)
model_full = AutoModelForSequenceClassification\
    .from_pretrained('bert-base-uncased', num_labels=2)
trainer_full = Trainer(model=model_full, args=args,
    train_dataset=tokenized['train'],
    eval_dataset=tokenized['validation'])
trainer_full.train()
```

```
# Вариант 2: Feature extraction (encoder заморожен)
model_frozen = AutoModelForSequenceClassification\
    .from_pretrained('bert-base-uncased', num_labels=2)
for param in model_frozen.bert.parameters():
    param.requires_grad = False # заморозка encoder
trainer_frozen = Trainer(model=model_frozen, args=args,
    train_dataset=tokenized['train'],
    eval_dataset=tokenized['validation'])
trainer_frozen.train()
```

Тема 3. PEFT: параметрически-эффективное дообучение

PEFT (Parameter-Efficient Fine-Tuning)

Семейство методов, позволяющих адаптировать предобученную модель к целевой задаче, обучая лишь 0.1–1% параметров. Критически важно для LLM, где full fine-tuning невозможен без кластера GPU.

LoRA (Low-Rank Adaptation, Hu et al., 2022)

Основной метод PEFT. Вместо обновления полной матрицы весов W ($d \times d$) обучаются две матрицы низкого ранга A ($d \times r$) и B ($r \times d$), где $r \ll d$ (обычно 4–64). Выход: $h = Wx + BAx$. Число обучаемых параметров: $2 \cdot d \cdot r$ вместо d^2 .

Adapters (Houlsby et al., 2019)

Вставка компактных MLP-блоков (down-project $\rightarrow$ nonlinearity $\rightarrow$ up-project) между слоями Transformer. Размер адаптера: $2 \cdot d \cdot \text{bottleneck}$. Обучаются только адаптеры.

Prefix Tuning (Li & Liang, 2021)

Обучаемые «виртуальные токены», конкатенируемые с входом на каждом слое attention. Не модифицирует веса модели.

QLoRA (Dettmers et al., 2023)

Комбинация 4-bit квантизации базовой модели и LoRA-адаптеров в FP16. Позволяет fine-tune LLaMA-65B на одной GPU 48 ГБ.

Тема 3. LoRA через библиотеку PEFT

КОД

```
from peft import LoraConfig, get_peft_model, TaskType
from transformers import AutoModelForSequenceClassification

# Загрузка базовой модели
model = AutoModelForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2)

# Конфигурация LoRA-адаптеров
lora_config = LoraConfig(
    task_type=TaskType.SEQ_CLS, # тип задачи
    r=8, # ранг матриц A и B
    lora_alpha=16, # масштабирующий коэффициент
    lora_dropout=0.1, # dropout в LoRA-слоях
    target_modules=['query', 'value'], # куда вставлять
)

# Оборачиваем модель в PEFT-обёртку
model = get_peft_model(model, lora_config)

# Статистика обучаемых параметров
model.print_trainable_parameters()
# Пример вывода: trainable params: 294,912 (0.27%)

# Далее обучение через стандартный Trainer
trainer = Trainer(model=model, args=args,
    train_dataset=tokenized['train'],
    eval_dataset=tokenized['test'])
trainer.train() # обучаются только LoRA-веса
```

Тема 3. Задача Б: LoRA vs Full Fine-tuning

ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Дообучите BERT-base на IMDB двумя способами: full fine-tuning и LoRA ($r=8$). Сравните ассигасу, число обучаемых параметров и время обучения.

```
from peft import LoraConfig, get_peft_model, TaskType
import time

# === Full fine-tuning: все параметры ===
model_full = AutoModelForSequenceClassification\
    .from_pretrained('bert-base-uncased', num_labels=2)
n_full = sum(p.numel() for p in model_full.parameters()
              if p.requires_grad)
print(f'Full FT params: {n_full/1e6:.1f}M')

t0 = time.time()
Trainer(model=model_full, args=args,
        train_dataset=tok_train,
        eval_dataset=tok_test).train()
time_full = time.time() - t0

# === LoRA: только адаптеры ===
model_lora = AutoModelForSequenceClassification\
    .from_pretrained('bert-base-uncased', num_labels=2)
model_lora = get_peft_model(model_lora, LoraConfig(
    task_type=TaskType.SEQ_CLS, r=8,
    lora_alpha=16, target_modules=['query', 'value']))
model_lora.print_trainable_parameters() # ~0.27%

t0 = time.time()
Trainer(model=model_lora, args=args,
        train_dataset=tok_train,
        eval_dataset=tok_test).train()
time_lora = time.time() - t0
```

Тема 4. Knowledge Distillation для NLP

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Knowledge Distillation (KD) в NLP — передача знаний из большой модели (teacher: BERT-large, GPT-3) в компактную (student: DistilBERT, TinyBERT). Специфика NLP: помимо logit-дистилляции применяются методы, учитывающие внутреннюю структуру Transformer.

DistilBERT (Sanh et al., 2019)

6-слойный student, обученный дистилляцией от 12-слойного BERT-base. Результат: 60% параметров удалены, 97% качества сохранено, inference в 1.6x быстрее. Loss: CE + KL-дивергенция + cosine embedding loss (выравнивание скрытых представлений).

MiniLM (Wang et al., 2020)

Дистилляция self-attention: выравнивание attention distributions последнего слоя teacher и student. Проще TinyBERT, но сопоставимое качество.

TinyBERT (Jiao et al., 2020)

Двухступенчатая дистилляция: (1) General distillation на неразмеченных данных — передача attention maps и hidden states; (2) Task-specific distillation — дообучение на целевой задаче с teacher-сигналом.

Для LLM

Дистилляция в малые модели (Phi, Gemma) через генерацию синтетических данных teacher-моделью.

Тема 4. Дистилляция с Hugging Face

КОД

```
import torch, torch.nn.functional as F
from transformers import (
    AutoModelForSequenceClassification, AutoTokenizer)

# Teacher: полный BERT-base (12 слоёв, 110M параметров)
teacher = AutoModelForSequenceClassification\
    .from_pretrained('textattack/bert-base-uncased-SST-2')
teacher.eval() # замораживаем teacher

# Student: DistilBERT (6 слоёв, 66M параметров)
student = AutoModelForSequenceClassification\
    .from_pretrained('distilbert-base-uncased', num_labels=2)

def distill_loss(s_logits, t_logits, labels, T=4, a=0.7):
    """Комбинированный loss: KL + CE."""
    # Мягкие распределения с температурой T
    kl = F.kl_div(
        F.log_softmax(s_logits/T, dim=-1),
        F.softmax(t_logits/T, dim=-1),
        reduction='batchmean') * (T*T)
    ce = F.cross_entropy(s_logits, labels)
    return a * kl + (1-a) * ce # баланс

# Цикл обучения student с teacher-сигналом
for batch in train_dl:
    with torch.no_grad():
        t_out = teacher(**batch).logits
    s_out = student(**batch).logits
    loss = distill_loss(s_out, t_out, batch['labels'])
    loss.backward(); optimizer.step()
```

Тема 4. Задача В: сравнение Teacher и Student

ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Сравните BERT-base (teacher) и DistilBERT (student) на SST-2. Измерьте ассурасу, число параметров, inference time на 1000 примерах.

```
from transformers import pipeline
import time

# Предобученные модели на SST-2 (sentiment)
models = {
    'BERT-base (teacher)':
        'textattack/bert-base-uncased-SST-2',
    'DistilBERT (student)':
        'distilbert-base-uncased-finetuned-sst-2-english',
}

for name, ckpt in models.items():
    # Создание inference pipeline
    clf = pipeline('sentiment-analysis', model=ckpt,
                  device=0) # GPU

    # Подсчёт параметров
    n_params = sum(p.numel() for p in
                  clf.model.parameters()) / 1e6

    # Замер inference time на 1000 текстах
    texts = dataset['test']['sentence'][:1000]
    t0 = time.time()
    preds = clf(texts, batch_size=32)
    latency = time.time() - t0

    print(f'{name}: {n_params:.0f}M params, '
          f'{latency:.2f}s for 1000 texts')
```

Тема 5. Pruning для Transformer-моделей

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Pruning в NLP имеет свою специфику, связанную со структурой Transformer. Помимо стандартного pruning весов, существуют методы структурного удаления компонентов архитектуры.

Attention Head Pruning (Michel et al., 2019)

Исследование показало, что в BERT-base до 40% attention heads можно удалить без потери качества на GLUE. Метод: оценка важности каждой головы через gradient-based importance score и последовательное удаление наименее значимых.

Movement Pruning (Sanh et al., 2020)

Обучаемые маски для весов. В отличие от magnitude pruning (удаление мелких весов), movement pruning удаляет веса, которые стремятся к нулю в процессе fine-tuning. Даёт лучшие результаты при высокой степени разреженности (>90%).

Layer Pruning

Удаление целых слоёв Transformer. Верхние слои специализируются под задачу, нижние — более универсальны. Для простых задач (sentiment) достаточно 3–6 слоёв вместо 12.

Block Sparsity

Структурированная разреженность блоками (2:4, 4:8), поддерживаемая аппаратно на NVIDIA Ampere GPU (до 2× ускорения).

Тема 5. Pruning attention heads в BERT



КОД

```
from transformers import BertForSequenceClassification
import torch

model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2)

# Оценка важности каждой attention head
# через gradient × activation (Taylor expansion)
head_importance = torch.zeros(12, 12) # layers × heads

model.eval()
for batch in eval_dl:
    outputs = model(**batch, output_attentions=True)
    loss = outputs.loss
    loss.backward()
    # Накопление importance scores
    for layer_idx in range(12):
        attn = model.bert.encoder.layer[layer_idx]\
            .attention.self
        # Gradient × Weight для каждой головы
        for head_idx in range(12):
            head_importance[layer_idx, head_idx] += \
                (attn.query.weight.grad.abs().sum()\
                 .item())

# Отсечение 30% наименее важных голов
threshold = head_importance.quantile(0.3)
heads_to_prune = {}
for l in range(12):
    heads = [h for h in range(12)
              if head_importance[l, h] < threshold]
    if heads: heads_to_prune[l] = heads

model.bert.prune_heads(heads_to_prune) # удаление голов
```

Тема 6. Quantization для NLP-моделей

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Квантизация — ключевой метод развёртывания LLM на доступном hardware. LLaMA-7B в FP32 занимает 28 ГБ, в INT8 — 7 ГБ, в INT4 — 3.5 ГБ.

Методы квантизации NLP-моделей:

bitsandbytes

Dettmers et al., 2022 — INT8 квантизация с mixed-precision decomposition. Разделяет матрицы на «нормальную» часть (INT8) и outlier-часть (FP16). Потеря качества < 0.1% на большинстве задач.

GPTQ

Frantar et al., 2023 — post-training квантизация до 4/3/2 бит с оптимальным округлением через решение задачи минимизации ошибки на калибровочном наборе. Основан на OBS (Optimal Brain Surgeon).

AWQ

Activation-Aware Weight Quantization, Lin et al., 2024 — масштабирование весов перед квантизацией пропорционально важности (определяемой по активациям). Превосходит GPTQ на 4-bit.

QLoRA

Dettmers et al., 2023 — 4-bit NormalFloat (NF4) квантизация базовой модели + LoRA-адаптеры в FP16 + Double Quantization (квантизация констант квантизации). Fine-tune LLaMA-65B на 1×A100.

Тема 6. INT8 и INT4 квантизация через bitsandbytes



КОД

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from transformers import BitsAndBytesConfig
import torch

# === INT8 квантизация: ~7 ГБ для LLaMA-7B ===
model_8bit = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Llama-2-7b-hf',
    load_in_8bit=True,      # INT8 через bitsandbytes
    device_map='auto'      # автораспределение по GPU
)

# === INT4 квантизация (NF4): ~3.5 ГБ для LLaMA-7B ===
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',      # NormalFloat4
    bnb_4bit_compute_dtype=torch.float16, # вычисления FP16
    bnb_4bit_use_double_quant=True   # double quantization
)
model_4bit = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Llama-2-7b-hf',
    quantization_config=bnb_config,
    device_map='auto'
)

# Проверка размера модели в памяти GPU
mem = torch.cuda.memory_allocated() / 1e9
print(f'GPU memory: {mem:.1f} GB')

# Генерация текста квантизированной моделью
tokenizer = AutoTokenizer.from_pretrained(
    'meta-llama/Llama-2-7b-hf')
```

Тема 6. Задача Г: QLoRA fine-tuning

ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Дообучите LLaMA-2-7B через QLoRA (4-bit + LoRA $r=16$) на датасете инструкций. Сравните с полной моделью по perplexity.

```
from peft import LoraConfig, get_peft_model
from transformers import BitsAndBytesConfig
from trl import SFTTrainer # Supervised Fine-Tuning

# 4-bit квантизация базовой модели
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_compute_dtype=torch.float16)

model = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Llama-2-7b-hf',
    quantization_config=bnb_config,
    device_map='auto')

# LoRA-адаптеры поверх квантизированной модели
lora_config = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=['q_proj', 'v_proj', 'k_proj', 'o_proj'],
    task_type='CAUSAL_LM')

model = get_peft_model(model, lora_config)
model.print_trainable_parameters() # ~0.1%

# Обучение через SFTTrainer (instruction tuning)
trainer = SFTTrainer(model=model, args=args,
    train_dataset=dataset['train'],
    dataset_text_field='text', max_seq_length=512)
trainer.train()
```

Тема 7. Эффективные механизмы внимания

Стандартный self-attention

Имеет сложность $O(n^2 \cdot d)$, где n — длина последовательности, d — размерность. Для $n=4096$ (типично для LLM) матрица attention занимает ~64 МБ на слой. Это ограничивает длину контекста.

Flash Attention (Dao et al., 2022)

Алгоритмическая оптимизация стандартного attention без приближений. Использует тайлинг (разбиение на блоки) и перевычисление для минимизации обращений к HBM (высокоуровневой памяти GPU). Результат: 2–4× ускорение, 5–20× экономия памяти. Точность идентична.

Flash Attention 2 (Dao, 2023)

Оптимизация параллелизма по длине последовательности, 2× быстрее Flash Attention 1.

Sparse Attention (Child et al., 2019)

Вычисление attention только для подмножества позиций (локальные + strided паттерны). $O(n \cdot \sqrt{n})$ сложность. Используется в GPT-3, BigBird, Longformer.

Linear Attention

Замена softmax на ядерные функции, позволяющая вычислять attention за $O(n \cdot d^2)$. Примеры: Performer (Choromanski et al., 2021), cosFormer. Точность может быть ниже стандартного attention.

Тема 7. Flash Attention 2 в PyTorch

код

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

# === Способ 1: Flash Attention через HuggingFace ===
# Автоматически используется, если установлен flash-attn
model = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Llama-2-7b-hf',
    torch_dtype=torch.float16,
    attn_implementation='flash_attention_2', # ключевой параметр
    device_map='auto'
)

# === Способ 2: PyTorch native SDPA (Scaled Dot-Product) ===
# Автоматически выбирает Flash Attention, если доступен
from torch.nn.functional import scaled_dot_product_attention

# Пример: ручной вызов эффективного attention
q = torch.randn(1, 8, 512, 64, device='cuda', dtype=torch.float16)
k = torch.randn(1, 8, 512, 64, device='cuda', dtype=torch.float16)
v = torch.randn(1, 8, 512, 64, device='cuda', dtype=torch.float16)

# PyTorch >= 2.0: автовыбор оптимального бэкенда
out = scaled_dot_product_attention(q, k, v) # Flash Attention
print(f'Output shape: {out.shape}') # [1, 8, 512, 64]

# Бенчмарк: Flash Attention vs стандартный attention
# Flash: ~2-4x быстрее, ~5-20x меньше памяти
```

Тема 7. Задача Д: бенчмарк attention-механизмов

ПРАКТИЧЕСКАЯ ЗАДАЧА

📄 **Задание:** Сравните стандартный attention и Flash Attention 2 для GPT-2 по памяти и скорости при длинах 512, 1024, 2048, 4096 токенов.

```
import torch, time
from transformers import AutoModelForCausalLM

results = []
for seq_len in [512, 1024, 2048, 4096]:
    for attn_impl in ['eager', 'flash_attention_2']:
        torch.cuda.empty_cache()
        torch.cuda.reset_peak_memory_stats()
        # Загрузка модели с указанным attention
        model = AutoModelForCausalLM.from_pretrained(
            'gpt2', torch_dtype=torch.float16,
            attn_implementation=attn_impl
        ).cuda()
        # Генерация входных данных
        ids = torch.randint(0, 50257, (1, seq_len)).cuda()
        # Замер: forward pass
        t0 = time.time()
        with torch.no_grad():
            for _ in range(10):
                model(ids)
        elapsed = (time.time() - t0) / 10
        mem = torch.cuda.max_memory_allocated() / 1e6
        results.append([seq_len, attn_impl,
            f'{elapsed*1000:.1f}ms', f'{mem:.0f}MB'])

import pandas as pd
print(pd.DataFrame(results,
    columns=['SeqLen', 'Attn', 'Latency', 'Memory']))
```

Тема 8. Аугментация текстовых данных

В отличие от CV, где аугментации (flip, crop, rotate) естественны, текстовые аугментации требуют сохранения семантики и грамматической корректности. Основные подходы:

Token-level

(1) **Synonym Replacement** — замена слов на синонимы из WordNet или word2vec. (2) **Random Insertion/Deletion/Swap** — случайные операции над токенами (Wei & Zou, 2019, EDA). Простые, но эффективные для малых датасетов (+1–3% accuracy).

Model-based

(3) **Back-Translation** — перевод текста на другой язык и обратно. Генерирует парафразы с сохранением смысла. (4) **Contextual Augmentation** — замена слов предсказаниями BERT Masked LM. Контекстуально корректные замены.

Generative

(5) **LLM-генерация** — использование GPT/Claude для создания синтетических обучающих примеров по инструкции. Масштабируемый подход для instruction tuning и few-shot задач.

 **Библиотеки:** nlpaug (универсальная), TextAttack (adversarial + augmentation), AugLy (Meta, мультимодальная). Ключевой принцип: аугментация не должна менять метку (label-preserving).

Тема 8. nlpaug: методы аугментации текста

Код

```
import nlpaug.augmenter.word as naw # pip install nlpaug
import nlpaug.augmenter.sentence as nas

text = 'Нейронные сети демонстрируют высокую точность'

# 1. Synonym Replacement через WordNet
aug_syn = naw.SynonymAug(aug_src='wordnet', lang='rus')
print(f'Synonym: {aug_syn.augment(text)}')

# 2. Contextual Augmentation через BERT Masked LM
aug_bert = naw.ContextualWordEmbsAug(
    model_path='bert-base-multilingual-cased',
    action='substitute', # замена слов
    aug_p=0.2)          # 20% слов заменяются
print(f'BERT: {aug_bert.augment(text)}')

# 3. Back-Translation (RU → EN → RU)
aug_bt = naw.BackTranslationAug(
    from_model_name='Helsinki-NLP/opus-mt-ru-en',
    to_model_name='Helsinki-NLP/opus-mt-en-ru')
print(f'BackTrans: {aug_bt.augment(text)}')

# 4. Random Insertion / Deletion / Swap (EDA)
aug_del = naw.RandomWordAug(action='delete', aug_p=0.1)
aug_swap = naw.RandomWordAug(action='swap', aug_p=0.1)
print(f'Delete: {aug_del.augment(text)}')
print(f'Swap: {aug_swap.augment(text)}')
```

Тема 8. Задача Е: эффект аугментации на BERT

📌 ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Обучите BERT на 500 примерах SST-2 без аугментации и с EDA (×4 аугментация). Сравните ассигасу — покажите эффект на малых данных.

```
import nlpaug.augmenter.word as naw
from datasets import load_dataset

# Берём только 500 примеров (имитация малых данных)
dataset = load_dataset('glue', 'sst2')
small_train = dataset['train'].select(range(500))

# EDA-аугментация: synonym + insertion + deletion
aug_syn = naw.SynonymAug(aug_src='wordnet')
aug_ins = naw.RandomWordAug(action='insert', aug_p=0.1)
aug_del = naw.RandomWordAug(action='delete', aug_p=0.1)

def augment_dataset(dataset, n_aug=3):
    """Генерация n_aug аугментаций для каждого примера."""
    augmented = {'sentence': [], 'label': []}
    for example in dataset:
        augmented['sentence'].append(example['sentence'])
        augmented['label'].append(example['label'])
        for aug in [aug_syn, aug_ins, aug_del]:
            augmented['sentence'].append(
                aug.augment(example['sentence'])[0])
            augmented['label'].append(example['label'])
    return augmented # 500 × 4 = 2000 примеров

aug_data = augment_dataset(small_train)
print(f'Исходных: 500, после аугментации: '
      f'{len(aug_data["sentence"])})')
```

Тема 9. Оптимизация токенизации и словаря

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Токенизация — критический, но часто недооценённый этап NLP-пайплайна. Выбор токенизатора определяет длину последовательностей (и, следовательно, вычислительную стоимость) и покрытие лексики.

BPE (Byte Pair Encoding)

Sennrich et al., 2016 — итеративное слияние частотных пар символов. Используется в GPT-2/3/4, RoBERTa. Словарь: 50 000 подслов. Преимущество: гибкий баланс между символьным и словесным уровнем.

WordPiece

Schuster & Nakajima, 2012 — аналог BPE, но с вероятностным критерием слияния (максимизация правдоподобия). Используется в BERT. Словарь: 30 000 подслов.

SentencePiece

Kudo & Richardson, 2018 — language-independent токенизация на уровне Unicode. Поддерживает BPE и Unigram LM. Используется в T5, LLaMA, XLNet. Работает без предварительной токенизации на пробелы.

Оптимизация: (1) Обучение домен-специфичного токенизатора для уменьшения fertility (среднего числа токенов на слово). (2) Выбор `max_length` — баланс между покрытием и скоростью. (3) Dynamic padding — padding до длины максимального примера в батче.

Тема 9. Обучение и оптимизация токенизаторов

КОД

```
from tokenizers import Tokenizer, models, trainers
from tokenizers import pre_tokenizers, decoders

# === Обучение BPE-токенизатора на своём корпусе ===
tokenizer = Tokenizer(models.BPE(unk_token='[UNK]'))
tokenizer.pre_tokenizer = pre_tokenizers.ByteLevel()
tokenizer.decoder = decoders.ByteLevel()

# Обучение на файлах корпуса
trainer = trainers.BpeTrainer(
    vocab_size=32000,      # размер словаря
    min_frequency=2,      # минимальная частота пары
    special_tokens=['[PAD]', '[UNK]', '[CLS]', '[SEP]', '[MASK]']
)
tokenizer.train(files=['corpus.txt'], trainer=trainer)
tokenizer.save('custom_tokenizer.json')

# === Сравнение fertility (токенов на слово) ===
from transformers import AutoTokenizer
text = 'Оптимизация нейросетевых архитектур'

for name in ['bert-base-multilingual-cased',
             'ai-forever/ruBert-base']:
    tok = AutoTokenizer.from_pretrained(name)
    tokens = tok.tokenize(text)
    fertility = len(tokens) / len(text.split())
    print(f'{name}: {len(tokens)} токенов, '
          f'fertility={fertility:.2f}')
```

Тема 9. Задача Ж: dynamic padding и оптимизация

ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Сравните обучение BERT с фиксированным padding (max_length=512) и dynamic padding. Измерьте время обучения и пиковое потребление памяти.

```
from transformers import DataCollatorWithPadding

# === Вариант 1: Фиксированный padding (max_length=512) ===
def tokenize_fixed(examples):
    return tokenizer(examples['text'],
                    padding='max_length', # все до 512 токенов
                    truncation=True, max_length=512)

dataset_fixed = dataset.map(tokenize_fixed, batched=True)
# Средний batch содержит ~80% padding-токенов!

# === Вариант 2: Dynamic padding (до макс. в батче) ===
def tokenize_dynamic(examples):
    return tokenizer(examples['text'],
                    truncation=True, max_length=512) # без padding

dataset_dynamic = dataset.map(tokenize_dynamic, batched=True)

# DataCollator добавит padding до макс. длины в батче
collator = DataCollatorWithPadding(tokenizer=tokenizer)

# Обучение с dynamic padding
trainer = Trainer(
    model=model, args=args,
    train_dataset=dataset_dynamic['train'],
    data_collator=collator) # ключевое отличие
trainer.train()

# Ожидаемый эффект: ~30-50% ускорение обучения
```

Тема 10. Развёртывание NLP-моделей в production

Финальный этап оптимизации — развёртывание (deployment) модели на сервере или edge-устройстве с минимальной latency и максимальным throughput.

ONNX Runtime

Кросс-платформенный inference-движок. Экспорт через optimum.onnxruntime (Hugging Face). Поддерживает graph-оптимизации: operator fusion, constant folding, layer/attention fusion. Ускорение: 2–3×.

TensorRT-LLM (NVIDIA)

Специализированный движок для LLM на GPU. Включает: in-flight batching, KV-cache paging, INT4/FP8 quantization, tensor parallelism. Ускорение: 3–8× по сравнению с PyTorch.

vLLM (Kwon et al., 2023)

Serving-фреймворк с PagedAttention (управление KV-cache как виртуальной памятью). Позволяет обслуживать множество пользователей с динамическим выделением GPU-памяти. Throughput в 3–24× выше наивного PyTorch serving.

Triton Inference Server (NVIDIA)

Production-ready сервер для батчированного inference с поддержкой HTTP/gRPC API.

Типичный pipeline для production LLM

Комбинирование: QLoRA → Merge LoRA → GPTQ Quantization → vLLM Serving.

Тема 10. Экспорт в ONNX и vLLM serving



КОД

```
# === Экспорт BERT в ONNX через optimum ===
# pip install optimum[onnxruntime]
from optimum.onnxruntime import ORTModelForSequenceClassification
from transformers import AutoTokenizer

# Экспорт предобученной модели в ONNX формат
model = ORTModelForSequenceClassification.from_pretrained(
    'distilbert-base-uncased-finetuned-sst-2-english',
    export=True # автоматический экспорт в ONNX
)
tokenizer = AutoTokenizer.from_pretrained(
    'distilbert-base-uncased-finetuned-sst-2-english')

# Inference через ONNX Runtime (2-3x быстрее PyTorch)
inputs = tokenizer('Great movie!', return_tensors='pt')
outputs = model(**inputs)
print(f'Prediction: {outputs.logits.argmax()}')

# === vLLM: high-throughput LLM serving ===
# pip install vllm
from vllm import LLM, SamplingParams

# Загрузка модели с автоматическим tensor parallelism
llm = LLM(model='meta-llama/llama-2-7b-chat-hf',
    quantization='awq', # AWQ квантизация
    tensor_parallel_size=1) # число GPU

params = SamplingParams(temperature=0.7, max_tokens=256)
outputs = llm.generate(['Explain NLP optimization'],
    params)
```

Тема 10. Ограничения и рекомендации

☐ ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

При оптимизации NLP-моделей необходимо учитывать практические ограничения.

- LoRA и PEFT значительно экономят ресурсы, но качество зависит от выбора `target_modules` и ранга `r`. Для сложных задач (multi-turn dialogue, reasoning) full fine-tuning может быть необходим.
- Квантизация 4-bit (GPTQ, AWQ) эффективна для inference LLM, но может давать деградацию на задачах, требующих точных числовых вычислений (code generation, math reasoning). INT8 — более безопасный выбор.
- Flash Attention ускоряет только forward/backward pass. Bottleneck может быть в data loading, tokenization или post-processing. Профилирование обязательно (torch.profiler, NVIDIA Nsight).
- Text Augmentation требует валидации: аугментированные примеры могут нарушать семантику, особенно при random augmentation.

Рекомендуемый pipeline: (1) Выбрать предобученную модель (HF Hub) → (2) LoRA/QLoRA fine-tuning → (3) Merge LoRA weights → (4) GPTQ/AWQ quantization → (5) vLLM или ONNX Runtime serving.

Ключевой принцип: профилировать каждый этап и оптимизировать bottleneck.

Итоги семинара

01

Hugging Face Transformers — стандартная экосистема для NLP

02

LoRA/QLoRA позволяют fine-tune LLM на одной GPU (0.1–1% параметров)

03

Knowledge Distillation: DistilBERT даёт 97% качества при 60% размера

04

Pruning attention heads — удаление 30–40% голов без потери качества

05

INT4/INT8 квантизация: LLaMA-7B помещается в 4–7 ГБ GPU-памяти

06

Flash Attention 2: ускорение в 2–4× без потери точности

07

Text Augmentation (EDA, Back-Translation) критичен для малых данных

08

vLLM + ONNX Runtime — production deployment с 3–24× ускорением

[HUGGINGFACE](#) [LORA](#) [PEFT](#) [BITSANDBYTES](#) [GPTQ](#)

[FLASH ATTENTION](#) [VLLM](#) [ONNX](#)